European Transport Research Review

RESEARCH Open Access

FleetPy: an open source simulator for reproducible research on mobility-on-demand services

Roman Engelhardt[1], Florian Dandl[1*], Arslan Ali Syed[1], Chenhao Ding[1], Santiago Alvarez-Ossorio Martinez[1], Yunfei Zhang[1], Hoda Hamdy[1], Joel Brodersen[1], Zhipu Chen[1,2] and Klaus Bogenberger[1]

Abstract

Mobility-on-demand (MoD) services have been extensively investigated across multiple research disciplines. For example, transport planners are interested in the impact on mode choice and traffic, while researchers in operations research focus on operational sub-problems such as assignment, pooling, and routing. These differing perspectives lead to varying setups for simulations and dynamic vehicle routing problems. Many developed approaches and algorithms are not open-source, and results are tied to specific problem settings, making fair comparisons difficult. To address this, we developed FleetPy, an agent-based simulator that provides a consistent and modular platform for studying and comparing MoD services. Its modular structure enables exploration of diverse aspects at varying levels of detail. These modules cover a wide range of fleet control tasks—including implementations of vehicle assignment, repositioning, pricing, and charging methods—alongside customer behavior models, network and routing components, and agent interactions. Well-designed interfaces facilitate coupling with established transport simulation frameworks and have been successfully tested in real-world experiments. FleetPy advances reproducible research in three ways: (1) enabling comparison of new algorithms with existing methods under consistent scenarios, (2) providing benchmark input data for Manhattan, Munich, and Chicago, and (3) supporting transport planners in developing MoD case studies with reduced technical effort.

Keywords Mobility-on-demand, Ride-pooling, Agent-based simulation, Open-source, Fleet operations, FleetPy

1 Introduction

Urban transportation systems face important challenges, including their contributions to environmental pollution and global warming, traffic congestion, and adverse effects on urban livability [1–3]. Mobility-on-Demand (MoD) services, where passengers can request rides via smartphone apps and are served on demand by a fleet of vehicles, offer a promising approach to tackling these issues. If implemented adequately, MoD services can help reduce car ownership and dependency, reduce on-street parking demand, and mitigate the environmental impacts of transportation. Furthermore, depending on the local context, MoD services may complement public transit services, improve accessibility to transportation, and provide economic benefits to users [4, 5].

Unlike traditional transport modes, MoD services are demand-driven, allowing dynamic adjustments of routes, departure times, and stop locations based on real-time user demand. This leads to complex interactions

*Correspondence:
Florian Dandl
florian.dandl@tum.de
[1]Chair of Traffic Engineering and Control, Technical University of Munich, Arcistrasse 21, 80333 Munich, Germany
[2]German Institute of Science & Technology, TUM Asia, 61 Nanyang Drive, Singapore 637335, Singapore

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

between supply (the fleet of vehicles) and demand (the user requests), as well as among users themselves, especially in ride-pooling scenarios that enable multiple users to share a ride. While analytical models can capture the big picture and produce managerial insights [6, 7], they often require simplifying assumptions, for example, ignoring dynamics and interactions, or heterogeneity of input variables [8]. Consequently, agent-based simulations have emerged as a valuable tool for estimating the social and economic impacts of MoD services, evaluating pricing schemes, policies, and regulations, and testing algorithms and operational strategies prior to real-world deployment, e.g. [9–14]. However, a simulation framework needs to specify a multitude of facets to evaluate these aspects of an MoD service [15].

Figure 1 provides a brief overview of the key aspects to be considered, including the input data, attributes of the simulation model, calculation of Key Performance Indicators (KPIs), and derivation of insights. Researchers face numerous design choices regarding the configuration of the fleet, the infrastructure and the simulation model, resulting in a large number of possible combinations for simulation configurations. We classify these decisions into five different categories: *service attributes*, *infrastructure*, *network*, *demand*, and *fleet control methods*.

Regarding the *service attributes*, the operator of the MoD service must decide on the number of vehicles and their corresponding types (e.g., electric or gas-powered, number of available seats, etc.). When human drivers are used, the operator may also want to consider dynamic fleet sizes to account for work shifts. The operator must also consider other service attributes such as time constraints for customer pick-up and drop-offs, pricing schemes, or maximum walking distances of customers to pick-up and drop-off locations. Additionally, the MoD *infrastructure* must be defined. This includes the city area where the service is operated, the locations for vehicle depots, charging or refueling facilities, or places where vehicle boardings are allowed. The latter also defines whether a service is offered door-to-door or asks its users to walk to or from a set of (virtual) stops.

Simulations also differ in their representation of *demand*. While most Operations Research studies focus on serving a given exogenous demand as good as possible, some agent-based transportation system models emphasize the traveler models, analyzing how these travelers respond to and are affected by MoD services [16–18]. In many real-world services, demand interaction occurs via an app, requiring operators to estimate and communicate the expected level of service to MoD users—such as anticipated pickup waiting times—even before a booking is made. However, this demand management aspect is only considered in few studies [19, 20]. Most studies simply treat an MoD request as a booking request, which might wait indefinitely (e.g. [10, 21]) or for a specific amount of time (e.g. [22]).

The *network* model defines how MoD vehicles move in the network, but also how the fleet control computes routing tasks to solve the underlying vehicle routing problem. While in the real world, the street network with its traffic states is a highly stochastic and dynamic system, various simplifications have been used for network

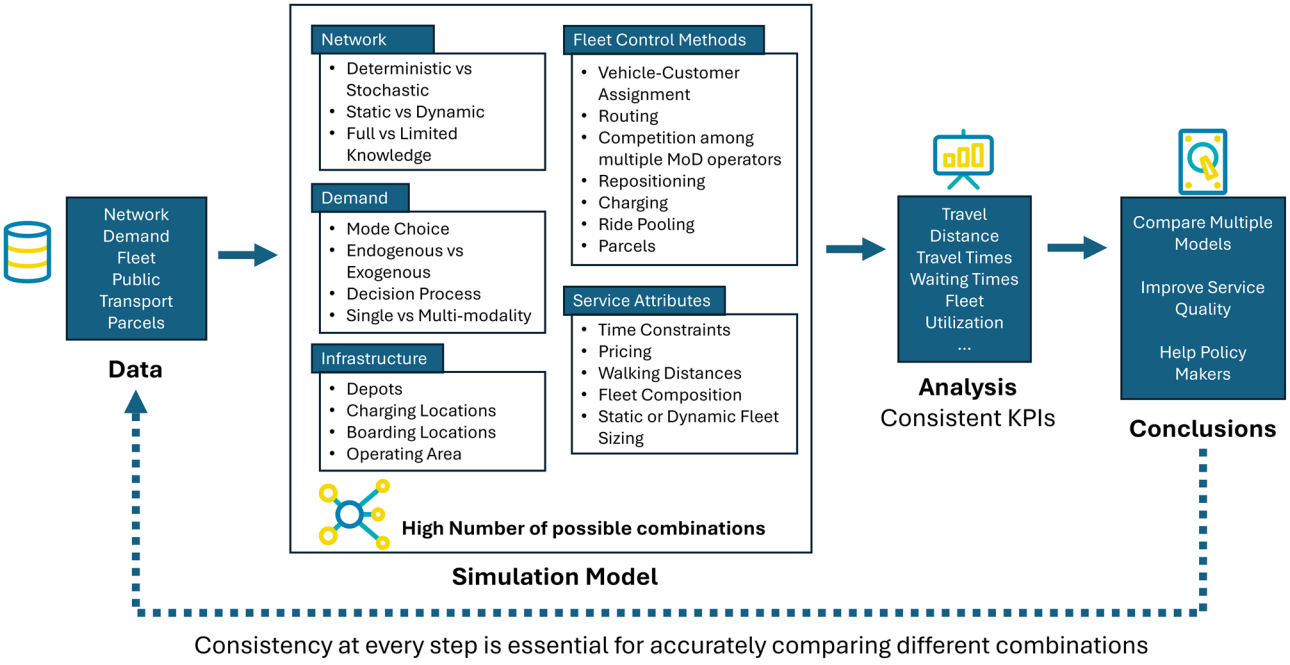


Fig. 1 Complexity of modeling and simulating MoD services

representation and solving computationally heavy routing tasks. Early studies represented vehicle movements in a purely geometric plane (e.g. [23–25]). More sophisticated approaches represent street networks as graphs, with edge-specific travel times capturing traffic conditions deterministically. These methods range from assuming constant daily travel times (e.g. [22]), to applying global scaling factors to simulate time-varying traffic states (e.g. [26, 27]), or using edge-specific dynamic and deterministic travel times (e.g. [28, 29]). To incorporate stochasticity and limited knowledge of the traffic states available to operators, some studies employ microscopic traffic simulators to evaluate the impact of uncertainty and dynamic traffic conditions [30, 31]. However, this microscopic resolution of traffic simulation can significantly increase the computation time, which is why mesoscopic simulations are often selected to model traffic impacts for MoD simulations [16–18].

The most important *fleet control* task of the fleet operator is the assignment of routes to vehicles, mainly consisting of stops to pick up and drop off users, as well as stops for other tasks such as repositioning and charging. In many cases, this problem is divided into routing between locations, user (re-)assignment (online or with reservation), vehicle repositioning and vehicle charging. Additionally, demand management (including pricing) is a critical task, as it directly affects the demand that needs to be served.

Within the MoD literature, many studies have developed algorithms for these tasks, often focusing on a specific task in a specific setting—defined by a combination of service attributes, infrastructure, network characteristics, demand models, and other control aspects.

In view of the wide range of above possibilities for MoD simulations, a collection of existing algorithms and a consistent simulation framework would benefit the wider research community (e.g., transport planners) as they could readily apply these algorithms to transportation system studies. We acknowledge that there is no *one size fits all* solution, as computational resources are typically a limiting factor in MoD simulation studies. Nonetheless, a modular simulation framework can partially address this challenge by allowing researchers to select modules based on the level of simulation resolution required for their specific research questions.

Beyond research on these aspects of MoD services, there has been increasing recognition in computational science and transportation research of the importance of reproducible simulation frameworks. Reproducibility challenges frequently arise due to heterogeneous datasets, undocumented preprocessing steps, non-public implementations, and inconsistent evaluation setups. These concerns extend beyond MoD research to transportation simulation more broadly, including macroscopic models, microscopic traffic simulation, and agent-based frameworks [32]. Recent work such as [33] highlights the lack of standardized practices in AMoD research and proposes structured approaches to improve transparency and comparability between studies. Although widely used simulation tools such as MATSim [16], SimMobility [17], Simulation of Urban MObility (SUMO) [34], and AMoDeus [35] provide powerful environments for transportation modeling, their application often relies on complex, study-specific input data that are not publicly available [36], while experimental setups are tightly coupled to underlying network models and simulation workflows. This not only limits reproducibility and cross-study comparability, but also constrains the flexibility of developers in exploring alternative problem formulations and algorithmic designs.

To address these broader challenges of reproducibility and comparability in MoD research, we have developed FleetPy, a modular, high-fidelity MoD fleet simulation framework that supports the implementation and sharing of code and algorithmic developments. To this end, FleetPy leverages standardized Base Classes and APIs to minimize redundant implementation effort, enabling researchers to focus exclusively on algorithm logic and facilitate seamless benchmarking under consistent conditions. We have already implemented several simulation flows and control algorithms (see next section) and recently published benchmark data sets (see Sect. 3).

FleetPy has evolved over the years from scientific work, with numerous studies building on its individual modules, and this paper presents the framework as a whole to highlight its support for reproducible research.

2 Methodology

The developed simulation framework, FleetPy, aims to model MoD services at a varying level of detail—from simple scenarios to complex, real-world applications. It is therefore imperative to consider detailed interactions between stakeholders and entities involved in the overall MoD system, and build clear abstract interfaces that allow modular integration. Figure 2 provides a high-level overview of modules and their interaction within the FleetPy simulation framework. The cornerstone of the framework is the MoD operator that implements different fleet control algorithms in various sub-modules. The MoD fleet moves on a street network, with the operator using a routing engine to calculate routes and estimate travel times and distances between network locations. Customer demand is managed by a dedicated demand module that also incorporates user behavior models.

The key interaction between demand (demand module) and supply (MoD Operator (s) and Public Transport Module) is handled via an abstract Mobility Broker Platform Module. Users request rides from available

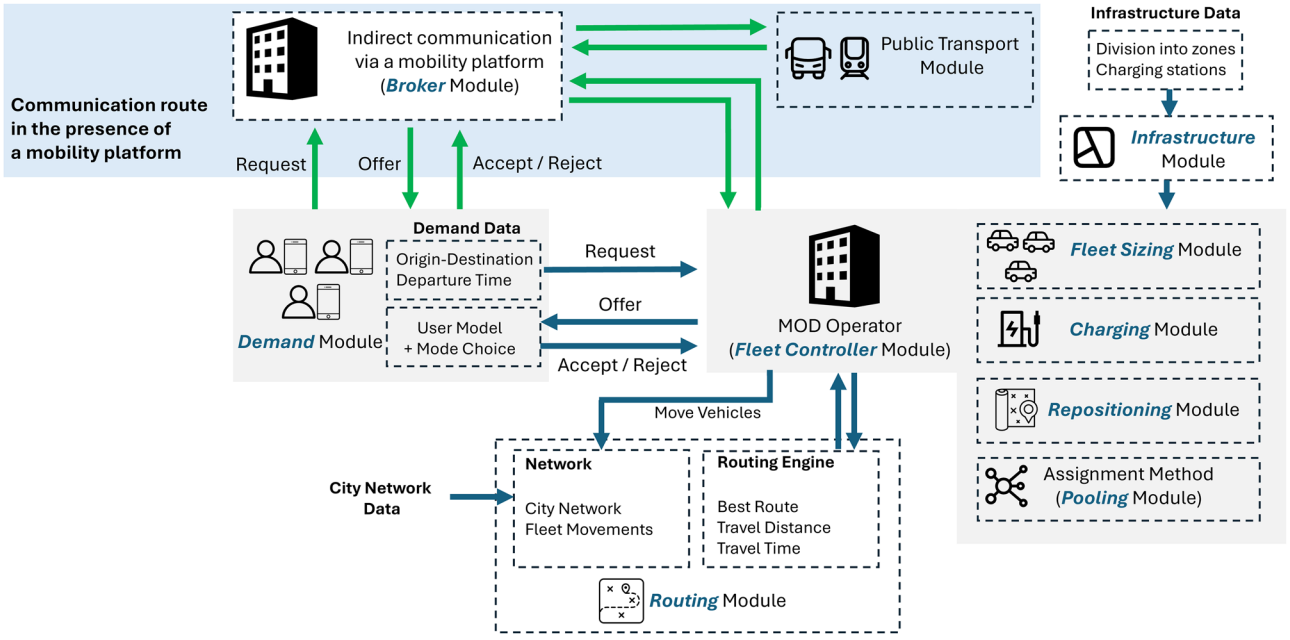


Fig. 2 Main components / modules and their interactions in the FleetPy framework

transport operators via this platform. Subsequently, users receive an offer that includes estimates of a trip, like estimated pick-up time, travel time, and fare. Based on the offers received, users choose to accept or reject certain mobility options. In the most basic implementation (represented by the blue arrows between demand and MoD operator in Fig. 2), this mobility platform feeds forward all communication between demand and supply, representing a direct communication of users and MoD providers in a free market or an unregulated MaaS platform. In other implementations of the mobility platform, this structure allows an integration of MoD services and public transport via intermodal trips, or regulated platforms to optimize the overall mobility system.

2.1 Simulation flow

A FleetPy simulation is discretized into time steps, typically with a resolution of a few seconds. It supports a customizable simulation flow, allowing FleetPy users to define the temporal sequence in which the interaction between different modules occurs.

There are currently two distinct simulation flows implemented in FleetPy, as illustrated in Fig. 3: Immediate Decision Simulation and Batch Offer Simulation. The Immediate Decision Simulation handles individual MoD user requests in order of their entry to the system. When a user submits a travel request, operators immediately generate service offers based on their current fleet state and request attributes. Once the user makes a decision on the offer(s), operators update the fleet assignment for that interaction. This process focuses on simulating personalized and immediate service engagements.

Conversely, the Batch Offer Simulation initially aggregates all new user travel requests in batches over a defined period. Then, operators perform centralized fleet optimization for these requests and dispatch service offers to all new users. This potentially enhances global optimization for vehicle allocation and resource scheduling through the collective evaluation of MoD benefits. The operators' fleet states are updated comprehensively after all users have responded to their offers. Theoretically, this leads to a better overall solution; however, in practice, an improved solution quality with batches is not guaranteed, especially when the users are modeled with the ability to reject MoD offers.

It should be noted that even in the Immediate Decision Simulation, the fleet control can use global re-optimization of user-vehicle assignments periodically, but only with the users who have already accepted the immediate offers.

2.2 Modules

As presented in Fig. 2, FleetPy is partitioned into multiple modules. Each module serves a specific role in MoD simulations and can be further divided into multiple sub-modules, forming a bottom-up modular architecture. Any MoD simulation can combine these modules in multiple ways according to the desired MoD service model. Additionally, most of the modules are structured in a way that they can be easily extended to incorporate new functionality or even work with third-party software via interfaces or by directly incorporating the modules as software packages. This section provides an overview of the individual modules, their associated functionalities,

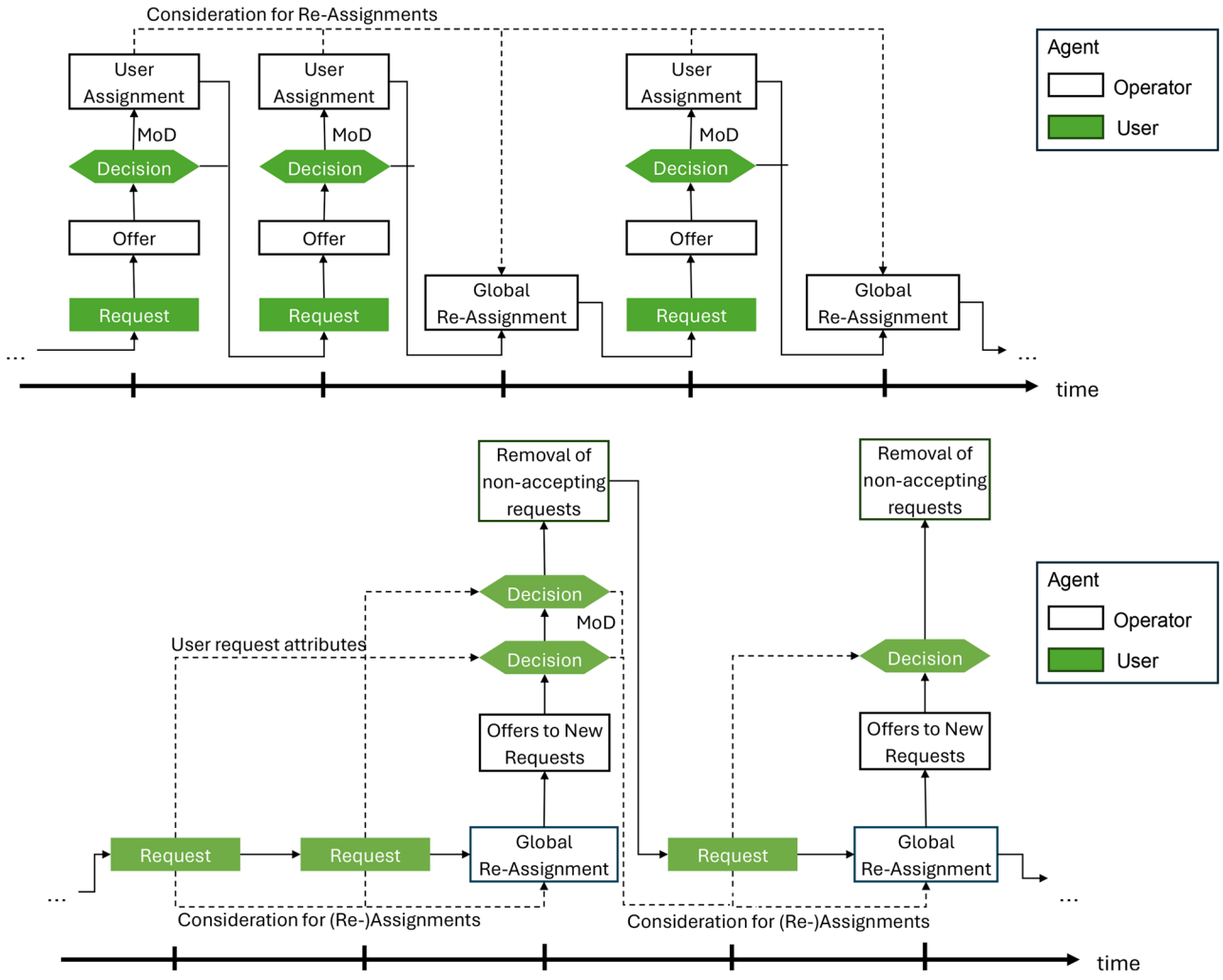


Fig. 3 Overview of the two simulation flows in FleetPy: Immediate Decision Simulation (top) and Batch Offer Simulation (bottom)

and the currently available implementations. The presented implementations are intended as exemplary solutions that have been used in the literature, rather than as definitive or optimal approaches. A comprehensive review and comparison of alternative algorithms is beyond the scope of this work; interested readers are referred to existing surveys, such as [33, 37], for a more detailed discussion.

2.2.1 Street network

FleetPy offers multiple *Network* modules that represent the road network and its traffic state. These modules mainly update the vehicle positions during the simulation and provide efficient routing methods to calculate paths and travel times between different locations.

In current implementations, the street network is represented as a directed graph with deterministic travel times associated with each edge. Travel times can be adjusted dynamically to represent varying load either by global travel time factors or by providing dynamic edge-specific travel time data. To incorporate stochasticity, interfaces with the microscopic traffic simulators AIMSUN and SUMO, and the mesoscopic traffic simulator of MATSim are implemented and will be published in a future version. The modular and transferable design of FleetPy allows identical MoD control approaches to be consistently implemented and tested across simulation environments with varying levels of detail.

Given the large number of routing queries required to solve vehicle routing problems, FleetPy incorporates efficient routing methods. Besides implementations in Python, shortest path computations based on Dijkstra's algorithm can also be called in C++ for computational efficiency by simply changing one scenario parameter. To further enhance computational efficiency, pre-processed travel time tables can be utilized.

2.2.2 Public transport

Independent of the street network, FleetPy incorporates a dedicated Public Transport (PT) module to simulate schedule-based public transport. This module constructs a corresponding public transport network based on the General Transit Feed Specification (GTFS), enabling querying for the fastest journey plans between two PT stations.

For fast and detailed routing queries, FleetPy currently features a C++ implementation of the Round-Based Public Transit Routing (RAPTOR) algorithm [38, 39]. Its modular design, however, also permits the integration of alternative routing methods as required.

2.2.3 Demand and user models

FleetPy models the MoD users in a separate Demand module. On an agent level, the module generates individual MoD users using an input data file for MoD requests. The file contains information required to generate MoD users, such as request-ID, the time the MoD request is made, and origin and destination locations.

Functionally, the Demand module also determines the accept/reject behavior of the users after they receive an MoD offer. FleetPy supports multiple user decision models and can be easily extended by custom models. In the basic model, the user simply accepts the first offer received, or leaves the system if all operators rejected the request or if the specified decision time limit has expired before receiving any offer.

More complex user models accept/reject or choose an offer (in case of multiple operators) based on the offer attributes such as waiting time, driving time, and the offered fare. With access to these attributes, user models accepting/rejecting based on hard constraints can be defined. The choice models can be further configured as desired (e.g., for logit-models) by creating custom implementations of the request module. To this end, homogeneous user attributes can be set in scenario configuration files or heterogeneous user-specific attributes can be added to the request files. Some examples of these attributes include the value of time or preprocessed utilities of other modes (e.g., walking, cycling or public transport). These can be used in actual choice models (e.g., a logit model) to determine which mode and/or which MoD operator should be selected by a specific traveler.

2.2.4 Multimodal trips

FleetPy supports the simulation of user journeys involving multiple modes of transport. Users can specify transit stops and the types of feeder services required, enabling the creation of more complex multimodal travel requests. Three feeder services are implemented in FleetPy: Utilizing MoD for first-mile, last-mile, or combined first-last-mile connections. For multimodal travel requests, the Broker platform deconstructs the user's request into multiple segments based on the modal state. It then generates corresponding sub-requests sent to the relevant MoD and PT operators. Finally, the offers for all sub-requests are consolidated into a single, comprehensive offer presented to the user.

2.2.5 Fleet control

The fleet control module simulates the behavior of MoD operators within the system. Its primary function is to dynamically assign schedules to vehicles in response to incoming demand and to generate trip offers based on the current fleet state. The operator's objective is typically to optimize a performance metric—–such as profit. The underlying task becomes a highly dynamic and stochastic vehicle routing problem. Given that this problem is NP-hard, optimal solutions are notoriously hard to find for large-scale instances. Therefore, the common strategy is to decompose the problem into smaller, more manageable sub-problems. FleetPy adopts this approach by organizing these sub-problems into distinct fleet control sub-modules, which are sketched in Figure 4 and described in the following sections.

2.2.5.1 Problem representation FleetPy implements three key objects to represent the vehicle routing problem to be solved: The *PlanRequest* represents the information the operator has of a customer request, which includes origin and destination locations, and time constraints for pick-up and drop-off. Feasible schedules (i.e., a set of possible vehicle schedules to serve a certain set of customers while adhering to all constraints) are represented by a *VehiclePlan*. A *VehiclePlan* further consists of an ordered list of *PlanStops* that encode stops at certain locations in the network where vehicles will perform various tasks, like boarding or alighting a certain set of customers. *VehiclePlans* are rated by an objective function. The goal of the control algorithms is to assign the best possible *VehiclePlans* to the vehicle fleet (at most one per vehicle) to maximize the operator's global objective.

The vehicles in the simulation will follow the *VehiclePlans*. However, due to uncertainties and properties unknown to the operator—such as stochastic travel times, or unexpected customer behavior—the *VehiclePlans* from prior time steps are periodically updated before any new assignments are made.

2.2.5.2 Assignment Solving the assignment problem is the core of the fleet control module. In the assignment problem, the best possible vehicle schedules are assigned to vehicles to serve currently active customer requests. The solution must also fulfill certain constraints, like time constraints on pick-up and drop-off or occupancy constraints for available vehicle seats.

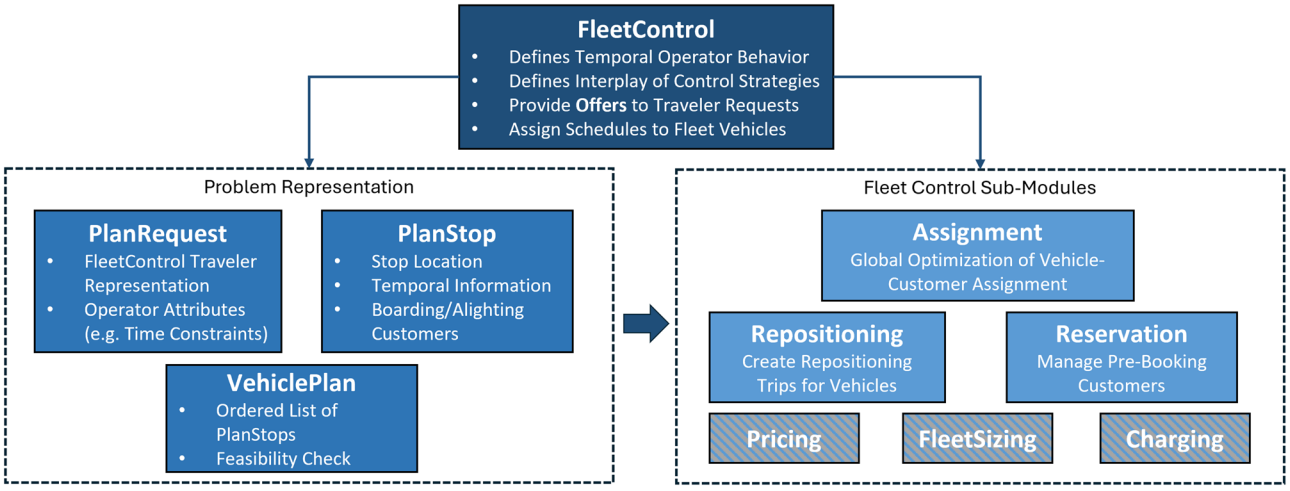


Fig. 4 Overview of the fleet control module

FleetPy implements multiple state-of-the-art algorithms to solve the assignment problem for ride-pooling. This includes a fast but sub-optimal insertion heuristic by Jaw et al. [40], a variant of the computationally heavy but anytime-optimal algorithm by Alonso-Mora et al. [22] (refer to Engelhardt et al. [41] for details), and the (heuristic linear assignment) algorithm developed by Simonetto et al. [42] representing a trade-off between optimality and computational complexity. Additional FleetPy modules expand the assignment problem further for other possible MoD configurations such as integration of parcel deliveries in Fehn et al. [43] and semi-on-demand services in Ng et al. [44, 45].

2.2.5.3 Reservations Usually, trip reservations can not be directly incorporated into the mentioned assignment algorithms due to a drastic increase in feasible vehicle schedules with advanced customer bookings. This significantly increases the solution space and, consequently, the computational load for finding a solution. In the extreme case of having all requests reserve their trip in advance, the problem of assigning these requests becomes a static vehicle routing problem.

Nevertheless, as the information evolves dynamically (e.g., travel times and additional online requests), it becomes necessary for the online assignment algorithms to regularly update the initial solutions that include must-be-served reservation requests. Therefore, multiple rolling-horizon based reservation strategies are implemented in FleetPy, as detailed in Engelhardt [37].

2.2.5.4 Repositioning As the assignment algorithm typically does not consider future demand when assigning schedules to the vehicle fleet, imbalances between supply and demand occur, with vehicles ending up in low-demand areas while absent from high-demand ones. Therefore, vehicle repositioning is applied to redistribute vehicles according to expected demand. In conjunction with different zone-based demand forecast methods, FleetPy provides implementations for multiple state-of-the-art repositioning strategies [22, 37, 46–48].

2.2.5.5 Offers When requesting trips from the operator, customers expect estimates on trip characteristics like expected pick-up and drop-off time and the fare for the trip. Customers use this information to decide whether to use the service or not.

Since customers expect quick replies and many may not use the service after requesting a trip, running the computationally intensive assignment algorithm for each request is usually not practical. FleetPy's standard implementation applies an insertion heuristic to find feasible assignments to estimate trip characteristics, while machine learning approaches are being developed to estimate characteristics of the MoD offer from current fleet states that do not require solving a vehicle routing problem [20].

The timing of the offer creation is determined by the simulation flow (see Fig. 3). In the Immediate Decision Simulation, offers must be generated as soon as a request is submitted. By contrast, the Batch Offer Simulation relaxes this requirement: here, the user model specifies how long a customer is willing to wait for an offer, for example through a predefined decision time limit.

2.2.5.6 Other modules FleetPy implements additional control modules that are only described here on a high level due to space limitations.

The charging module ensures that the vehicles maintain sufficient battery levels during their operation. In the current implementation, the vehicle is redirected to the nearest available charging station when its state of charge

(SOC) falls below a predefined threshold. Additionally, FleetPy incorporates dynamic pricing strategies to determine trip prices based on surge factors. The utilization-based strategy adjusts the fares based on the current fleet utilization. To model dynamically changing fleet sizes (to represent the impacts of driver shifts), the time-based fleet sizing strategy activates or deactivates vehicles to align with a specified fleet size over time.

2.3 Simulation inputs

To start a simulation, FleetPy requires key datasets including the network, travel demand, and configuration files. The exact number of input parameters depends on the complexity of the simulated scenario (e.g., a simulation with multiple operators at least requires multiple fleet size parameters). Most of the parameters remain constant over a study. Hence, as sketched in Fig. 5, configuration files are divided into two parts: constant configuration for all studies and a scenario-specific configuration file, which gives a quick overview of varying input parameters.

Our GitHub repository https://github.com/TUM-VT/FleetPy provides example configurations, a scenario creator GUI and several further explanations on how to set up scenarios. The available modules and the structure of input data are also described in the GitHub Wiki.

2.4 Data output

At the completion of each simulation, multiple output files are generated containing key performance indicators relevant to different stakeholder groups. At the most disaggregate level, the *user stats* file provides detailed information about each request, including origin and destination nodes, request time, and the service outcome—such as whether an offer was generated and accepted, fare, pick-up and drop-off times and locations, among others. Similarly, the *operational stats* file captures each action performed by every MoD vehicle (boarding, alighting, driving), along with the corresponding timestamp, location, and list of on-board requests.

Additionally, FleetPy produces two *standard evaluation* files with aggregated performance metrics. One file summarizes indicators per MoD vehicle (e.g., total distance driven, emissions, operating costs), while the other reports system-level metrics, such as the percentage of accepted requests, total distance traveled, average travel and waiting times, vehicle occupancy, share of empty kilometers, and total emissions. The framework also enables users to easily compute additional indicators tailored to their specific use case.

3 Benchmark data set

While it is possible to set up simulations for arbitrary case studies, Open-Source data sets are available in FleetPy format that allow FleetPy users to readily simulate and compare results. The input data for Manhattan, NY [49], Chicago, Illinois [50], and Munich, Germany [51] will be described at a high level in the following, while the interested reader is referred to [37] for further details.

3.1 Manhattan, NYC

Due to the publicly available taxi data set [52], Manhattan, New York City, is a popular case study to evaluate control algorithms of MoD services (e.g. [22, 25, 46]).

The network for this data set is extracted from OpenStreetMap (OSM). Overall, the network consists of $4,410$ nodes and $9,574$ edges. Customers can only start and end their trip at specific access nodes, as we do not allow boarding at nodes that are connected to edges labeled as motorway or are unlabeled. Edges and access nodes are shown in Fig. 6b.

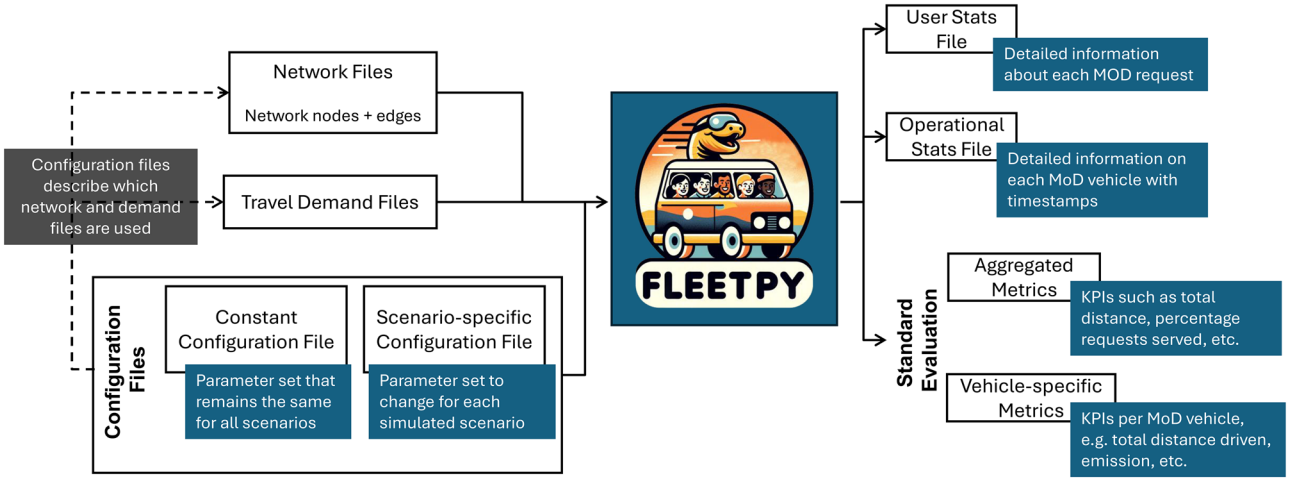
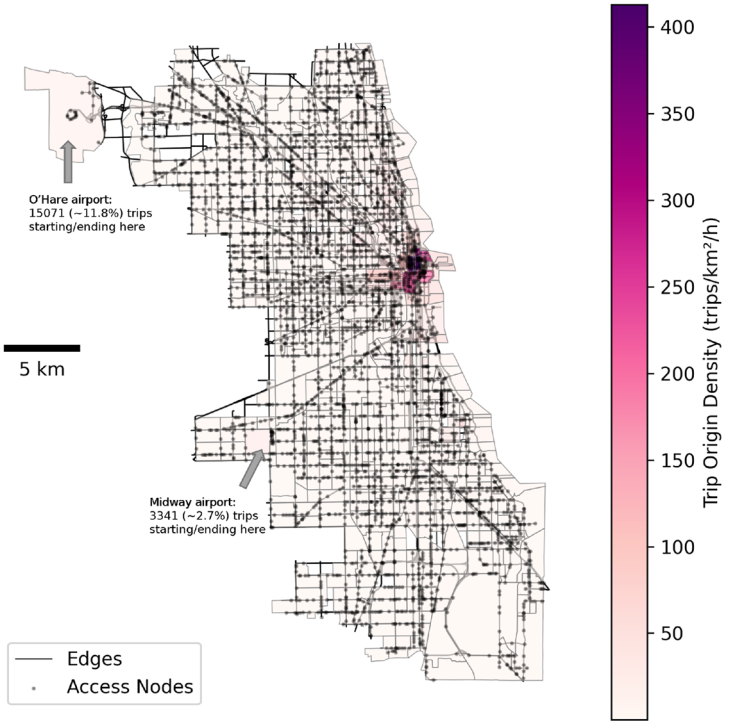
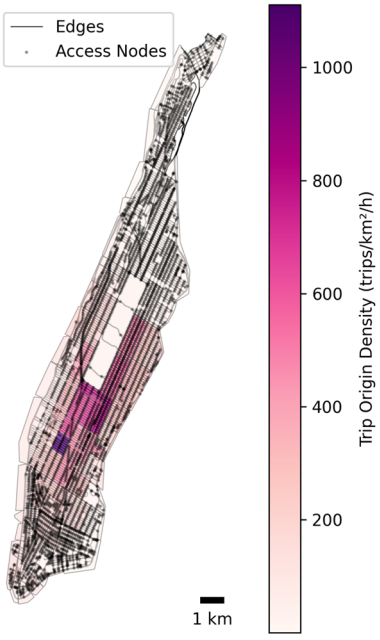


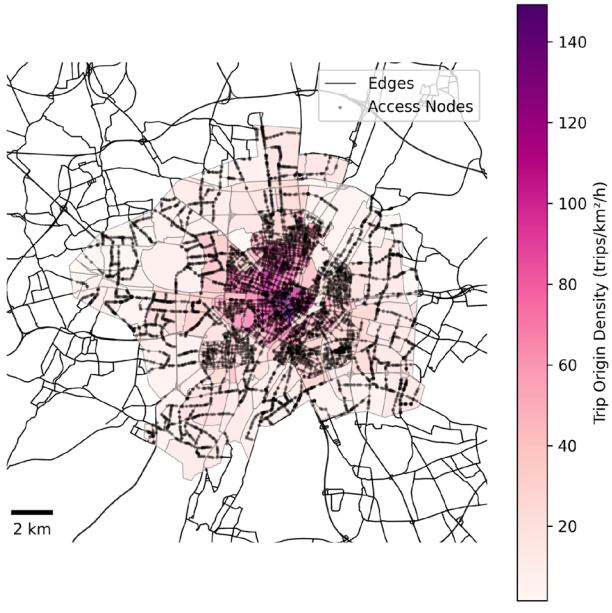
Fig. 5 The standard input and output files for a FleetPy simulation



(a) Chicago.



(b) Manhattan.



(c) Munich (10% PV Demand).

Fig. 6 Street networks, access nodes and spatial demand distribution of request origins for the different vase studies. Zonal aggregation refers to census tracts, taxi zones and municipalities for Chicago, Manhattan and Munich, respectively

The taxi data set for the week of 11/11/2018 to 11/18/2018 is used to create demand files. After the filtering of presumably false data (see [37]) and only keeping the trips that start and end in Manhattan, $1,807,255$ trips remain.

To calibrate network travel times, the reported trip duration in the data set is compared to the travel time of the fastest path when considering the maximum allowed speed from the OSM data on each edge. For each hour of the day, a scaling factor is calculated for the fastest path travel times to resemble the reported travel times of all trips in the data set in the same hour on average.

Next to daily files, subsamples of 10%, 25%, and 50% demand penetration of the original data set are created.

3.2 Chicago, Illinois

The street network for Chicago is also extracted from OSM. To reduce the size of the network, edges labeled as "residential" or "living_street" are removed from the network, resulting in $12,585$ nodes with $27,446$ edges. Customers can only start and end their trip at specific access nodes. All nodes with adjacent edges not labeled as "primary", "secondary", "tertiary", or unlabeled edges are not considered access nodes. Due to the size of the network, the set of access nodes is further reduced by randomly removing access nodes if another access node can be found within a distance of 300m. This procedure is repeated until $4,000$ access nodes are left, resulting in a number small enough to preprocess travel time lookup tables between those nodes to reduce the computational time needed for routing queries. Figure 6a shows the resulting network with all access nodes.

The MoD Demand is generated using the publicly available Transportation Network Companies (TNC) data set for Chicago, Illinois [53] for the week of 06/06/2022 to 06/12/2022. After filtering out the faulty data, $1,055,772$ trips remain. The data set includes daily demand files, and subsamples of 10%, 25%, and 50% demand penetration of the original data set.

Moreover, travel time scaling factors are calculated using a similar approach as for the Manhattan data set.

3.3 Munich, Germany

The final data set is Munich, Germany. In contrast to Manhattan and Chicago, no publicly available taxi or TNC data set is available. Although Uber or taxi services generally operate in Munich, the mode share of TNC and taxi services is significantly lower than in the US. Therefore, this data would be of limited use for evaluating large-scale MoD services (especially considering future autonomous services).

Instead, a synthetic demand data set is created based on private vehicle trips in Munich. Network and demand data are based on a microscopic traffic simulation model of Munich, described in [54]. After exporting the network from the microscopic traffic simulation model, the input network includes $35,474$ and $64,888$ edges. The relevant part for the simulation in this network is shown in Fig. 6c. The figure also shows the assumed operating area for creating demand data, which spans around Munich's city center and major parts of the city.

The calibrated microscopic traffic simulation model includes private vehicle trip origin-destination (OD) matrices for a typical weekday in Munich from 2016 for each hour of the day. These matrices are used to create trip requests. Entries in the OD matrices are used as Poisson rates for the number of requests between each OD zone pair. Up to 15% of the private vehicle trips are converted to MoD customer requests, which results in a total of around $104,000$ requests depending on the random seed for the Poisson process.

Access nodes are defined as all intersections within the operating area not connected to road sections with a speed limit higher than 50km/h to prevent boarding on major roads. The resulting 5113 access nodes are shown in Fig. 6c.

Network travel times are extracted from the microscopic traffic simulation model as well. After running the traffic simulation model, the average travel time for vehicles to pass each edge is extracted for each hour of the day.

Subsamples of 1%, 2%, 5%, 10%, and 15% demand penetration of the original OD matrices are created. Five random seeds are used to sample from the OD matrices for each demand penetration level.

3.4 Comparison of the case studies

Table 1 compares the main characteristics of the input data for the three case studies. The three case study areas differ considerably in scale and demand characteristics. Chicago has by far the largest operating area, while Manhattan shows the highest demand density due to its small size and high trip volume. Average trip lengths increase

Table 1 Comparison of main characteristics of the input data for the three case studies

Case Study	Operating Area	No. Trips*	Trip Density	Avg. Trip Length	Avg. Speed**
Munich	$175\,\mathrm{km}^2$	$\sim 104,000$	$\sim 24.8 \frac{1}{\mathrm{km}^2\mathrm{h}}$	6.0 km	$34.6 \frac{\mathrm{km}}{\mathrm{h}}$
Manhattan	$59\,\mathrm{km}^2$	213,996	$151 \frac{1}{\mathrm{km}^2\mathrm{h}}$	2.9 km	$16.1 \frac{\mathrm{km}}{\mathrm{h}}$
Chicago	$627\,\mathrm{km}^2$	127,528	$8.47 \frac{1}{\mathrm{km}^2\mathrm{h}}$	9.2 km	$31.7 \frac{\mathrm{km}}{\mathrm{h}}$

with area size, but congestion in Manhattan results in travel speeds about half those in Munich and Chicago.

Figure 6 shows the street networks, access nodes, and spatial demand distribution of request origins for the different case studies. The map of Chicago in Fig. 6a shows the large spatial spread of the operating area. Trip origins show a pronounced hotspot in the city center, with lower densities in the surrounding areas. A closer examination of the data reveals two additional hotspots, which are less visible in Fig. 6a due to the large size of the corresponding census tract zones. These are located around O'Hare International Airport in the northwest and Midway International Airport in the southwest. O'Hare accounts for roughly 12% of all trips, while Midway contributes about 3%, both representing notable shares of total demand.

Figure 6b illustrates the elongated island of Manhattan with its network and demand distribution. Unlike Chicago, Manhattan does not exhibit multiple distinct hotspots; instead, demand is concentrated around Midtown and the southern edge of Central Park. The highest densities occur near Penn Station and Grand Central Terminal, two major transport hubs, each exceeding 1,000 requests per square kilometer per hour. Demand gradually decreases toward both the northern and southern ends of the island.

Figure 6c presents the street network, access nodes, and spatial distribution of request origins for the Munich case study. Unlike the U.S. cities, Munich's street network lacks a grid pattern, instead exhibiting an organic structure with high road density in the city center and sparser networks toward the outskirts. The demand distribution is similarly centered around the city core, with lower densities in surrounding areas. Compared to Chicago and Manhattan, however, the decline in demand density toward the outskirts is less pronounced, and overall demand is lower.

Overall, the three case studies capture distinct urban mobility contexts, enabling the evaluation of MoD services in FleetPy across diverse settings. Munich reflects a typical European city with moderate demand density concentrated in the city center and a gradual decline toward the outskirts. Manhattan represents a highly dense urban environment characterized by exceptionally high demand density and short trip lengths. Chicago, by contrast, illustrates a large metropolitan area with lower overall demand density but pronounced hotspots, particularly in the city center and at its airports.

4 Exemplary applications

For this study, we used FleetPy version 1.0.2. We simulate two one-day ride-pooling scenarios using the Manhattan benchmark dataset described in Sect. 3.1, based on a 10% sample of trip requests (21,400 requests) from 12 November 2018. Key simulation (input) parameters include a simulation time step of 60s, a maximum relative detour time of 40%, a maximum waiting time of 480s, and a re-optimization interval of 60s. Boarding and alighting times are set to 30s per passenger. The simulations can be reproduced by following the steps described in our github repository: https://github.com/TUM-VT/FleetPy/tree/1.0.1/studies/paper_2025_applications

4.1 Batch assignment

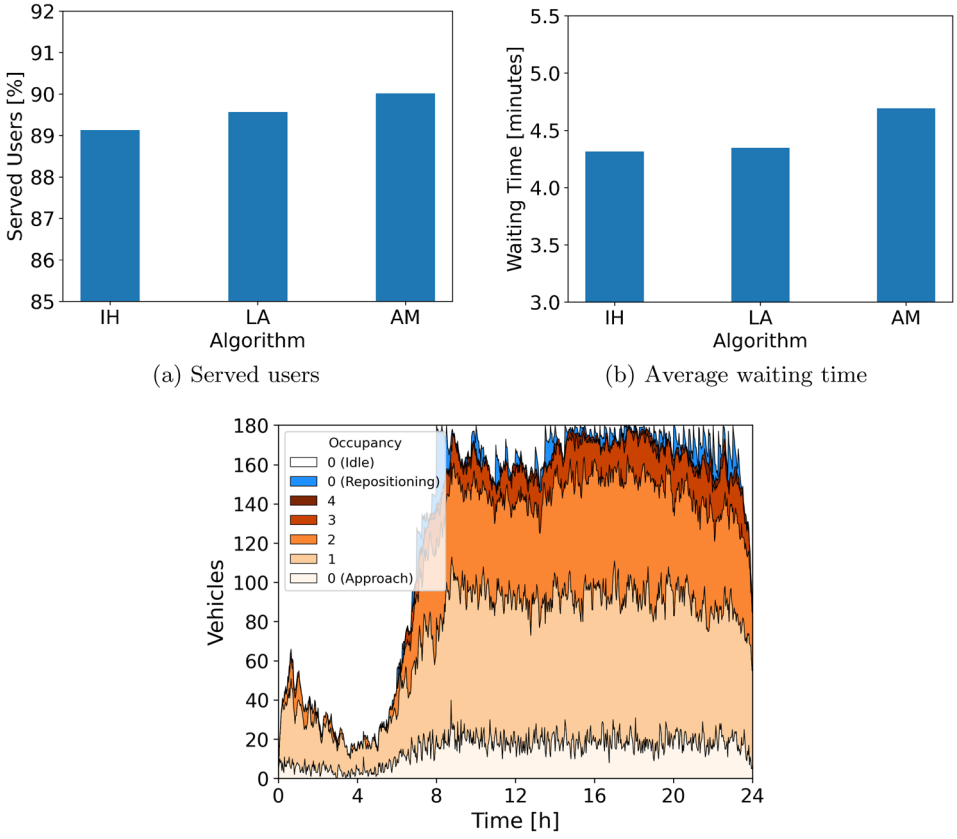
In the first scenario, we compare three batch assignment strategies: an insertion heuristic (IH) [40], the linear assignment approach (LA) [42], and the Alonso-Mora algorithm (AM) [22]. The vehicle fleet consists of 180 vehicles, each with a capacity of four passengers. Figure 7 illustrates the performance of these strategies in terms of user service rate (Fig. 7a) and average waiting time (Fig. 7b). Among the three, the AM approach achieves the highest percentage of served users (90%), followed by LA (89.6%) and IH (89%). The average waiting time is highest for AM at 4.7 minutes, while IH and LA report 4.32 and 4.35 minutes, respectively. This reflects a common trade-off in batch assignment: strategies that achieve higher service rates may do so at the cost of longer waiting times. Figure 7c shows the temporal stacked occupancy rates of the vehicle fleet under the AM strategy. Empty vehicles with zero occupancy are either idle, approaching customers, or repositioning. The average occupancy rate for all vehicles throughout the day is approximately 1.33, compared to 1.31 for both IH and LA. The marginally higher occupancy under AM suggests more efficient vehicle utilization, despite longer wait times.

4.2 Multi-operator

Next, we examine a scenario with two MoD service providers: the first operator offers a hailing service (180 vehicles), whereas the second operator offers a pooling service (120 vehicles) at a discounted fare. The hailing service provides direct, non-shared rides with a maximum of one passenger per vehicle, fixed fares, and minimal detours. The pooling service supports shared rides of up to four passengers per vehicle, flexible routing, and discounted fares. Passenger mode choice is determined by a utility function:

$$U(\lambda) = -\lambda\, t_{\text{wait}} - \lambda\, t_{\text{drive}} - \text{fare}$$

where λ denotes the value of time (VOT), and user heterogeneity is captured by three representative VOT levels: $\lambda \in \{0.25,\ 0.45,\ 0.75\}$. Low-VOT users are more cost-sensitive and willing to accept shared rides and detours, while high-VOT users prioritize travel time and are more likely to choose hailing services. Pooling fare discounts ranged from 0% to 50% in 5% increments. For each discount rate and each VOT cohort,



(a) Served users

(b) Average waiting time

(c) Temporal stacked plot for occupancy rate for AM

Fig. 7 Comparison of batch assignment strategies

separate simulations are conducted and the modal split is recorded for both services.

Figure 8a shows that deeper discounts drive a steady rise in pooling modal share alongside a decline in hailing modal share. Discounts between 10 and 30% yield the greatest gains in pooling uptake. Crucially, the adoption crossover, defined as the discount at which pooling modal share exceeds hailing modal share, shifts to higher values as user time sensitivity increases: approximately 2% for $\lambda = 0.25$, 3% for $\lambda = 0.45$, and 5% for $\lambda = 0.75$. In other words, more time-sensitive users require progressively larger discounts before they switch to shared rides. Figure 8b shows that operator profit generally declines with increasing discount rates. While the pooling service initially benefits from discounts, excessive fare reductions lead to diminishing returns in profitability for both service types.

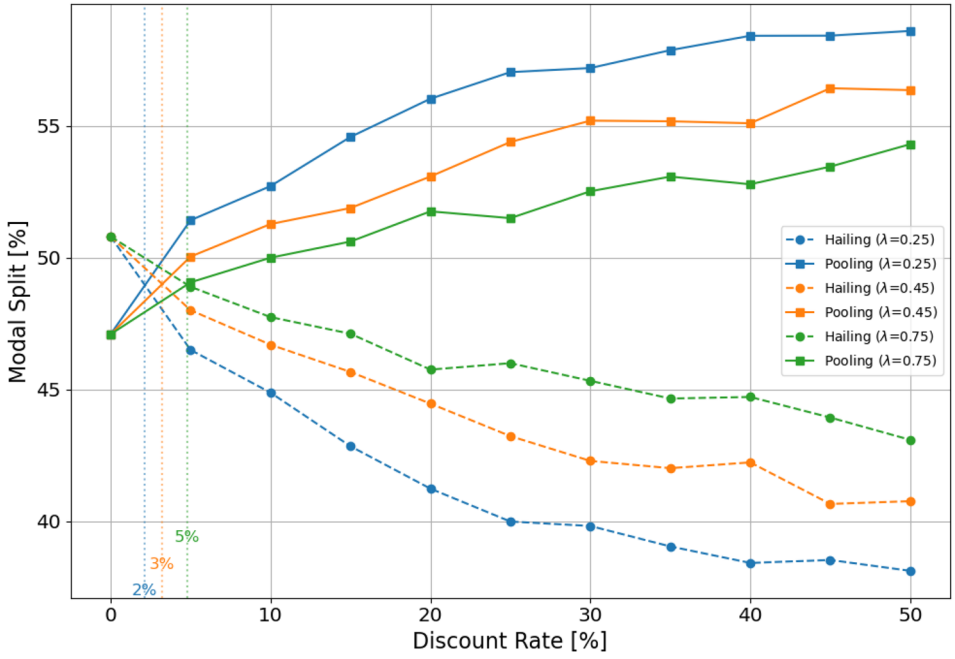
5 How to generate reproducible research with FleetPy

When a user wants to publish their FleetPy simulation study, they should set up all simulations within a single study directory and describe their study in a Readme.md file. Then they can use the script "zip_studies.py" to pack all relevant information. The script will first scan all scenario configuration files to identify all used data inputs. In addition to the scenario configuration and required input data files, evaluation scripts in the studies-directory and the FleetPy source code are included in the zip. Moreover, a user can select if the result files should be added to the zip. This zip file can then be published alongside the paper.

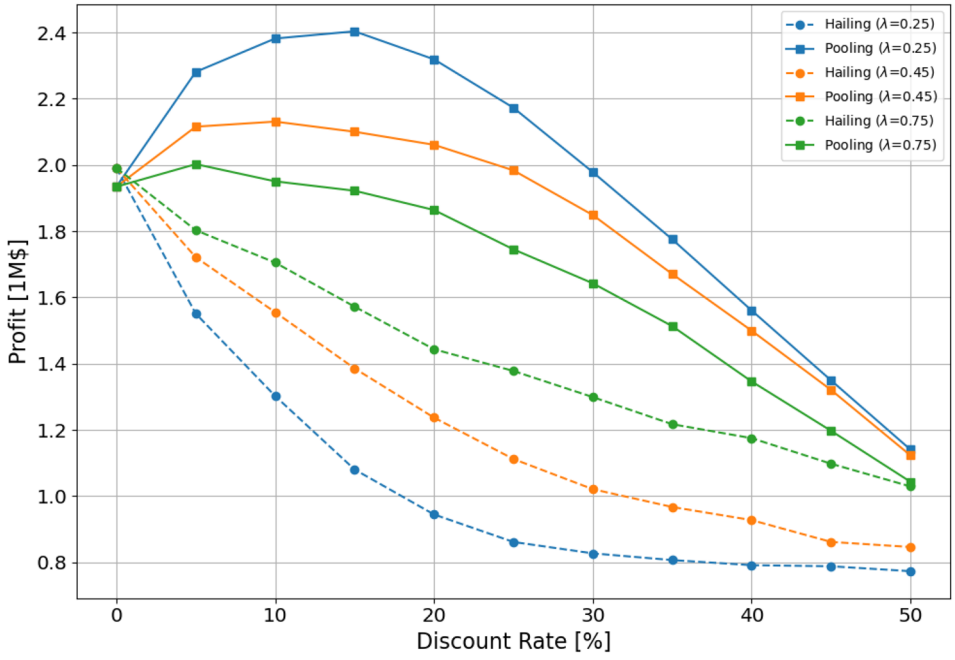
For this paper, we only created a very small exemplary study so that we could add it to our github repository. It can be found in the studies directory as paper_2025_applications https://github.com/TUM-VT/FleetPy/tree/main/studies/paper_2025_applications. Within this directory, the Readme.md describes how to run the simulations and the jupyter notebook can be used to reproduce the result plots once the simulations are finished. Furthermore, we ran our zip_study.py script and uploaded the study data under the following DOI: http://doi.org/10.14459/2026mp1846790.

6 Conclusion

Research about MOD services is a very active field. Since service and study designs are often very specific and not standardized, many studies lack benchmarks and results



(a) Modal shares reflect the percentage of users served by each operator



(b) Operator profits

Fig. 8 Impact of pooling and fare discounts on user mode choices and operator profit using three value-of-time (VOT) cohorts

are not easily transferable. To address this, we provide a highly modular open-source simulation framework FleetPy, which allows to model street network, public transport, demand, and even simulation flow according to the needs of a specific study.

We provide standardized input data for Manhattan, Chicago, and Munich, as well as open-source fleet control algorithms, which enable reproducibility of results within the studied settings. Moreover, these algorithms can be applied to different contexts and serve as benchmarks. For instance, one can use various repositioning

and assignment algorithms in new case studies, explore different service designs with varying waiting times, capacity, or detour constraints, or even simulate electric vehicles with charging requirements if desired—most of these different simulations only require changes to input data or the simulation configuration files.

We also present illustrative results from two small case studies. The first compares different user-assignment algorithms and provides numerical evidence that global assignment optimization can serve more users than linear assignment or greedy insertion heuristics; however, at the cost of slightly longer waiting times. In the second case study, we compare two competing MoD services: one offering only solo trips and the other providing pooled trips at a discounted fare. We then analyze the impact of the discount rate on user split and operator profits.

In the future, we will release connections of FleetPy to other open-source traffic simulators such as MATSim and SUMO. While the inherent demand model in FleetPy can already model other modes, such as biking, their utilities often have to be preprocessed, and their movement is modeled by teleportation. Moreover, the inherent demand model is not suited to consider the full complexity of trip chains. Hence, we also plan to connect FleetPy to MATSim to have the best of both worlds: FleetPy modeling the digital world, with user communication and operational algorithms, and MATSim simulating the physical world with traffic and demand models. Moreover, we plan to extend FleetPy in several directions. We will look closely at the assignment module with more realistic modeling of user interactions, such as non-instant user decisions. Additionally, we plan to extend the framework to be suitable for machine-learning applications such as machine-learning guided optimization or reinforcement learning. Beyond technical extensions, a key focus for our future development is to continuously enhance the accessibility of FleetPy for new developers by expanding our library of step-by-step tutorials and interactive notebooks, thereby further supporting the reproducible research objectives of this framework.

We invite all researchers to test and use our framework. Our open-source code models stochastic processes in mobility-on-demand services in a reproducible way (with random seeds). This paper provides instructions for conducting reproducible research with FleetPy and demonstrates it through an exemplary study. We are open to feedback and collaborations. We are also happy to feature your FleetPy related data or research papers in the FleetPy repository https://github.com/TUM-VT/FleetPy/wiki/8_Featured_Studies.

Acknowledgements
We want to acknowledge prior contributors to FleetPy: Fabian Fehn, Fynn Wolf, Arjan de Ruijter, and Max Ng.

Author contributions
Conceptualization: RE, FD, AS, KB. Methodology: RE, FD, AS. Software: RE, FD, AS, CD, SAOM, YZ, HH, JB, ZC. Formal analysis: RE, FD, HH, ZC. Data curation: RE, FD. Writing - original draft: RE, FD, AS, CD, SAOM, YZ, HH, JB, ZC. Writing - review: KB. Visualization: all authors. Supervision: FD, KB.

Funding
Open Access funding enabled and organized by Projekt DEAL.

Data availability
FleetPy is available at https://github.com/TUM-VT/FleetPy and we used version 1.0.2 in our case study. The input data are from the FleetPy Manhattan data set https://zenodo.org/records/15187906. Finally, a description on how to run the presented exemplary applications and the respective simulation configuration files can be found here: https://github.com/TUM-VT/FleetPy/tree/1.0.1/studies/paper_2025_applications.

Declarations

Competing interests
The authors declare no competing interests.

Received: 29 August 2025 / Accepted: 7 July 2026
Published online: 15 July 2026

References

1. Shiftan, Y., Ben-Akiva, M., Jong, G., Hakkert, S., & Simmonds, D. (2002). Evaluation of externalities in transport projects. *European Journal of Transport and Infrastructure Research*, *2*(4). https://doi.org/10.18757/ejtir.2002.2.4.3728.
2. Santos, G., Behrendt, H., Maconi, L., Shirvani, T., & Teytelboym, A. (2010). Part i: Externalities and economic policies in road transport. *Research in Transportation Economics*, *28*(1), 2–45. https://doi.org/10.1016/j.retrec.2009.11.002.
3. Schröder, D., Kirn, L., Kinigadner, J., Loder, A., Blum, P., Xu, Y., & Lienkamp, M. (2023). Ending the myth of mobility at zero costs: An external cost analysis. *Research in Transportation Economics*, *97*, 101246. https://doi.org/10.1016/j.retrec.2022.101246.
4. Shaheen, S. A., Cohen, A. P., & Zohdy, I. (2016). *Shared mobility: Current practices and guiding principles*. U.S. Department of Transportation, Federal Highway Administration, Washington, DC. https://rosap.ntl.bts.gov/view/dot/42193.
5. Cats, O., Kucharski, R., Danda, S. R., & Yap, M. (2022). Beyond the dichotomy: How ride-hailing competes with and complements public transport. *PLoS One*, *17*(1), 1–17. https://doi.org/10.1371/journal.pone.0262496.
6. Herminghaus, S. (2019). Mean field theory of demand responsive ride pooling systems. *Transportation Research Part A: Policy and Practice*, *119*, 15–28. https://doi.org/10.1016/j.tra.2018.10.028.
7. Zhang, K., & Nie, Y. (2021). To pool or not to pool: Equilibrium, pricing and regulation. *Transportation Research Part B: Methodological*, *151*, 59–90. https://doi.org/10.1016/j.trb.2021.07.001.
8. Bilali, A., Engelhardt, R., Dandl, F., Fastenrath, U., & Bogenberger, K. (2020). Analytical and agent-based model to evaluate ride-pooling impact factors. *Transportation Research Record: Journal of the Transportation Research Board*, *2674*(6), 1–12. https://doi.org/10.1177/0361198120917666.
9. Hörl, S., Ruch, C., Becker, F., Frazzoli, E., & Axhausen, K. W. (2019). Fleet operational policies for automated mobility: A simulation assessment for zurich. *Transportation Research Part C: Emerging Technologies*, *102*, 20–31. https://doi.org/10.1016/j.trc.2019.02.020.
10. Hyland, M., & Mahmassani, H. S. (2020). Operational benefits and challenges of shared-ride automated mobility-on-demand services. *Transportation Research Part A: Policy and Practice*, *134*, 251–270. https://doi.org/10.1016/j.tra.2020.02.017.
11. Kaddoura, I., Bischoff, J., & Nagel, K. (2020). Towards welfare optimal operation of innovative mobility concepts: External cost pricing in a world of shared autonomous vehicles. *Transportation Research Part A: Policy and Practice*, *136*, 48–63. https://doi.org/10.1016/j.tra.2020.03.032.
12. Dandl, F., Engelhardt, R., Hyland, M., Tilg, G., Bogenberger, K., & Mahmassani, H. S. (2021). Regulating mobility-on-demand services: Tri-level model and bayesian optimization solution approach. *Transportation Research Part C: Emerging Technologies*, *125*, 103075. https://doi.org/10.1016/j.trc.2021.103075.

13. Li, S., Yan, Z., & Wu, C. (2021). Learning to delegate for large-scale vehicle routing. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. S. Liang, & J. W. Vaughan (Eds.), *Advances in neural information processing systems* (Vol. 34, pp. 26198–26211). Curran Associates, Inc, Red Hook, NY, USA. https://doi.org/10.5555/3540261.3542267.
14. Gurumurthy, K. M., & Kockelman, K. M. (2022). Dynamic ride-sharing impacts of greater trip demand and aggregation at stops in shared autonomous vehicle systems. *Transportation Research Part A: Policy and Practice*, *160*, 114–125. https://doi.org/10.1016/j.tra.2022.03.032.
15. Psaraftis, H. N., Wen, M., & Kontovas, C. A. (2016). Dynamic vehicle routing problems: Three decades and counting. *Networks*, *67*(1), 3–31. https://doi.org/10.1002/net.21628.
16. Horni, A., Nagel, K., & Axhausen, K. W. (2016). *The multi-agent transport simulation MATSim*. London: Ubiquity Press. https://doi.org/10.5334/baw.
17. Adnan, M., Pereira, F. C., Azevedo, C. M. L., Basak, K., Lovric, M., Raveau, S., Zhu, Y., Ferreira, J., Zegras, C., & Ben-Akiva, M. (2016). Simmobility: A multi-scale integrated agent-based simulation platform. In *95th Annual Meeting of the Transportation Research Board* (Vol. 2). The National Academies of Sciences, Engineering, and Medicine, Washington, DC. https://www.researchgate.net/publication/296673237_SimMobility_A_Multi-Scale_Integrated_Agent-based_Simulation_Platform.
18. Auld, J., Hope, M., Ley, H., Sokolov, V., Xu, B., & Zhang, K. (2016). Polaris: Agent-based modeling framework development and implementation for integrated travel demand and network and operations simulations. *Transportation Research Part C: Emerging Technologies*, *64*, 101–116. https://doi.org/10.1016/j.trc.2015.07.017.
19. Fleckenstein, D., Klein, R., & Steinhardt, C. (2023). Recent advances in integrating demand management and vehicle routing: A methodological review. *European Journal of Operational Research*, *306*(2), 499–518. https://doi.org/10.1016/j.ejor.2022.04.032.
20. Ding, C., Dandl, F., & Bogenberger, K. (2025). Predicting offers for mobility-on-demand services: A machine learning-based approach. In *104th Annual Meeting of the Transportation Research Board*. https://mediatum.ub.tum.de/node?id=1765899.
21. Riley, C., Legrain, A., & van Hentenryck, P. (2019). Column generation for real-time ride-sharing operations. In H. Rousseau (Ed.), *Integration of constraint programming, artificial intelligence, and operations research. Lecture notes in computer science* (Vol. 11494, pp. 472–487). Springer International Publishing, Cham. https://doi.org/10.1007/978-3-030-19212-9_31.
22. Alonso-Mora, J., Samaranayake, S., Wallar, A., Frazzoli, E., & Rus, D. (2017). On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment. *Proceedings of the National Academy of Sciences*, *114*(3), 462–467. https://doi.org/10.1073/pnas.1611675114.
23. Fagnant, D. J., & Kockelman, K. M. (2014). The travel and environmental implications of shared autonomous vehicles, using agent-based model scenarios. *Transportation Research Part C: Emerging Technologies*, *40*, 1–13. https://doi.org/10.1016/j.trc.2013.12.001.
24. Hyland, M., & Mahmassani, H. S. (2018). Dynamic autonomous vehicle fleet operations: Optimization-based strategies to assign avs to immediate traveler demand requests. *Transportation Research Part C: Emerging Technologies*, *92*, 278–297. https://doi.org/10.1016/j.trc.2018.05.003.
25. Dandl, F., Hyland, M., Bogenberger, K., & Mahmassani, H. S. (2019). Evaluating the impact of spatio-temporal demand forecast aggregation on the operational performance of shared autonomous mobility fleets. *Transportation*, *46*(6), 1975–1996. https://doi.org/10.1007/s11116-019-10007-9.
26. Markov, I., Guglielmetti, R., Laumanns, M., Fernández-Antolín, A., & Souza, R. (2021). Simulation-based design and analysis of on-demand mobility services. *Transportation Research Part A: Policy and Practice*, *149*, 170–205. https://doi.org/10.1016/j.tra.2021.01.004.
27. Dandl, F., Hyland, M., Bogenberger, K., & Mahmassani, H. S. (2020). Dual-horizon forecasts and repositioning strategies for operating shared autonomous mobility fleets. In *99th Annual Meeting of the Transportation Research Board*. https://mediatum.ub.tum.de/1543181.
28. Engelhardt, R., Dandl, F., Bilali, A., & Bogenberger, K. (2019). Quantifying the benefits of autonomous on-demand ride-pooling: A simulation study for munich, Germany. In *2019 IEEE Intelligent Transportation Systems Conference (ITSC)* (pp. 2992–2997). IEEE, Auckland, New Zealand. https://doi.org/10.1109/ITSC.2019.8916955.
29. Syed, A. A., Zhang, Y., & Bogenberger, K. (2023). Data-driven spatio-temporal scaling of travel times for AMoD simulations. In *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)* (pp. 3583–3588). IEEE, Bilbao, Spain. https://doi.org/10.1109/ITSC57777.2023.10422313.
30. Gueriau, M., Cugurullo, F., Acheampong, R. A., & Dusparic, I. (2020). Shared autonomous mobility on demand: A learning-based approach and its performance in the presence of traffic congestion. *IEEE Intelligent Transportation Systems Magazine*, *12*(4), 208–218. https://doi.org/10.1109/MITS.2020.3014417.
31. Wolf, F., Engelhardt, R., Zhang, Y., Dandl, F., & Bogenberger, K. (2023). Effects of dynamic and stochastic travel times on the operation of mobility-on-demand services. In *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)* (pp. 5476–5481). IEEE, Bilbao, Spain. https://doi.org/10.1109/ITSC57777.2023.10422554.
32. Riehl, K., Kouvelas, A., & Makridis, M. A. (2025). Revisiting reproducibility in transportation simulation studies. *European Transport Research Review*, *17*(1), 22. https://doi.org/10.1186/s12544-025-00718-9.
33. Li, X., Alharbi, M., Gammelli, D., Harrison, J., Rodrigues, F., Schiffer, M., Pavone, M., Frazzoli, E., Zhao, J., & Zardini, G. (2026). Reproducibility in the control of autonomous mobility-on-demand systems. *IEEE Transactions on Robotics*, *42*, 1428–1447. https://doi.org/10.1109/TRO.2026.3666153.
34. Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P., & Wießner, E. (2018). Microscopic traffic simulation using sumo. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)* (pp. 2575–2582). IEEE.
35. Ruch, C., Horl, S., & Frazzoli, E. (2018). Amodeus, a simulation-based testbed for autonomous mobility-on-demand systems. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)* (pp. 3639–3644). IEEE, Macau, China. https://doi.org/10.1109/ITSC.2018.8569961.
36. Hörl, S., & Balac, M. (2021). Synthetic population and travel demand for Paris and Île-de-france based on open and publicly available data. *Transportation Research Part C: Emerging Technologies*, *130*, 103291. https://doi.org/10.1016/j.trc.2021.103291.
37. Engelhardt, R. (2024). Autonomous ride-pooling-services in urban environments: Operational strategies and simulation. PhD thesis, Technische Universität München. https://mediatum.ub.tum.de/1755168.
38. Delling, D., Pajor, T., & Werneck, R. F. (2015). Round-based public transit routing. *Transportation Science*, *49*(3), 591–604. https://doi.org/10.1287/trsc.2014.0534.
39. Rabelo, M. (2025). RAPTOR, 1c528c08899bfdf44adda3ff5faf3764c5b767a5, GitHub repository. https://github.com/mariaarabelo/RAPTOR/tree/1c528c08899bfdf44adda3ff5faf3764c5b767a5.
40. Jaw, J.-J., Odoni, A. R., Psaraftis, H. N., & Wilson, N. H. M. (1986). A heuristic algorithm for the multi-vehicle advance request dial-a-ride problem with time windows. *Transportation Research Part B: Methodological*, *20*(3), 243–257. https://doi.org/10.1016/0191-2615(86)90020-2.
41. Engelhardt, R., Dandl, F., & Bogenberger, K. (2020). *Speed-up heuristic for an on-demand ride-pooling algorithm*. https://doi.org/10.48550/arXiv.2007.14877.
42. Simonetto, A., Monteil, J., & Gambella, C. (2019). Real-time city-scale ridesharing via linear assignment problems. *Transportation Research Part C: Emerging Technologies*, *101*, 208–232. https://doi.org/10.1016/j.trc.2019.01.019.
43. Fehn, F., Engelhardt, R., Dandl, F., Bogenberger, K., & Busch, F. (2023). Integrating parcel deliveries into a ride-pooling service—an agent-based simulation study. *Transportation Research Part A: Policy and Practice*, *169*, 103580. https://doi.org/10.1016/j.tra.2022.103580.
44. Ng, M. T. M., Engelhardt, R., Dandl, F., Volakakis, V., Mahmassani, H. S., & Bogenberger, K. (2024). Semi-on-demand off-peak transit services with shared autonomous vehicles – service planning, simulation, and analysis in Munich, Germany. https://doi.org/10.48550/arXiv.2408.10547.
45. Ng, M. T. M., Engelhardt, R., Dandl, F., Mahmassani, H. S., & Bogenberger, K. (2024). Semi-on-demand transit feeders with shared autonomous vehicles and reinforcement-learning-based zonal dispatching control. *IEEE*. https://doi.org/10.1109/ITSC58415.2024.10920214.
46. Zhang, R., & Pavone, M. (2016). Control of robotic mobility-on-demand systems: A queueing-theoretical perspective. *The International Journal of Robotics Research*, *35*(1–3), 186–203. https://doi.org/10.1177/0278364915581863.
47. Wallar, A., van der Zee, M., Alonso-Mora, J., & Rus, D. (2018). Vehicle rebalancing for mobility-on-demand systems with ride-sharing. In *IROS Madrid 2018* (pp. 4539–4546). IEEE, Piscataway, New Jersey. https://doi.org/10.1109/IROS.2018.8593743.
48. Syed, A. A., Dandl, F., Kaltenhäuser, B., & Bogenberger, K. (2021). Density based distribution model for repositioning strategies of ride hailing services. *Frontiers in Future Transportation*, *2*, 681451. https://doi.org/10.3389/ffutr.2021.681451.
49. Engelhardt, R., & Dandl, F. FleetPy: Input data for Manhattan case study. https://doi.org/10.5281/zenodo.15187906.

50. Dandl, F., & Engelhardt, R. FleetPy: Input data for Chicago case study. https://doi.org/10.5281/zenodo.15189440.
51. Engelhardt, R., & Dandl, F. FleetPy: Input data for Munich case study. https://doi.org/10.5281/zenodo.15195726.
52. City of New York. (2024). TLC trip record data - TLC. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page Accessed 20.02.2024.
53. Chicago Department of Business Affairs & Consumer Protection. (2022). Chicago TNC data. https://data.cityofchicago.org/Transportation/Transportation-Network-Providers-Trips-2022/2tdj-ffvb Accessed 31.07.2023.
54. Dandl, F., Bracher, B., & Bogenberger, K. (2017). Microsimulation of an autonomous taxi-system in munich. In *2017 5th IEEE International Conference on Models and Technologies for Intelligent Transportation Systems (MT-ITS)* (pp. 833–838). https://doi.org/10.1109/MTITS.2017.8005628.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.